

Week 42: From Backpropagation to Differential Equations

Morten Hjorth-Jensen
Viktor Svensson

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for today

First hour

1. **Backpropagation**: the chain rule, organised
2. **Activation and cost functions**: why they come in pairs
3. **Universal approximation**: what a network can represent

Second hour

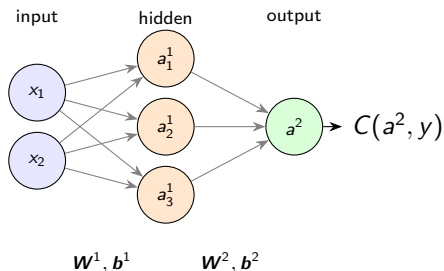
4. **Neural networks for differential equations**
 - ▶ the equation as the cost
 - ▶ why the activation function matters
 - ▶ boundary conditions
 - ▶ pros and cons vs. standard methods

On the board (take notes)

Most derivations today are done on the blackboard. The slides give the setup and the results.

Tuesday: backpropagation in code. **Exercises week 42**: deadline Friday October 16.

Recap: the forward pass



One sample $\mathbf{x}$, layers $l = 1, \dots, L$, $\mathbf{a}^0 = \mathbf{x}$:

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l, \quad \mathbf{a}^l = f(\mathbf{z}^l)$$

- ▶ $\mathbf{W}^l$ has shape (nodes out, nodes in)
- ▶ f acts element by element; the last layer has its own *output* activation
- ▶ the cost C compares the output $\mathbf{a}^L$ with the target $\mathbf{y}$

In code with many samples we stack them as rows,
 $\mathbf{Z}^l = \mathbf{A}^{l-1} \mathbf{W}^l + \mathbf{b}^l$: the same equations, transposed.

The four equations of backpropagation

We want $\partial C / \partial w_{jk}^l$ and $\partial C / \partial b_j^l$ for every layer. Key quantity: the *error* of node j in layer l ,

$$\delta_j^l = \frac{\partial C}{\partial z_j^l}.$$

On the board (W1)

Use the chain rule to find δ^L , then δ^l from δ^{l+1} , then the gradients. Note: node j in layer l feeds *every* node in layer $l + 1$.

The four equations of backpropagation

We want $\partial C / \partial w_{jk}^l$ and $\partial C / \partial b_j^l$ for every layer. Key quantity: the *error* of node j in layer l ,

$$\delta_j^l = \frac{\partial C}{\partial z_j^l}.$$

On the board (W1)

Use the chain rule to find δ^L , then δ^l from δ^{l+1} , then the gradients. Note: node j in layer l feeds every node in layer $l + 1$.

Result ($\circ$: element-wise product)

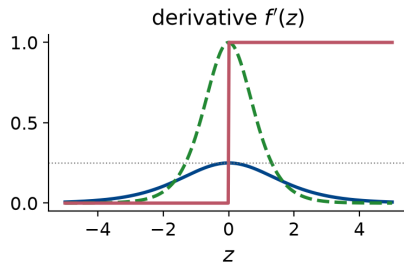
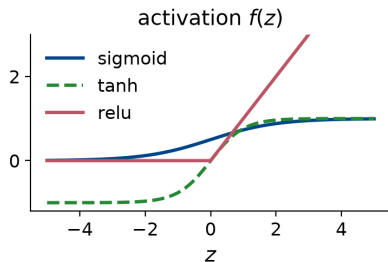
$$\delta^L = f'(z^L) \circ \frac{\partial C}{\partial a^L}$$

$$\delta^l = ((W^{l+1})^T \delta^{l+1}) \circ f'(z^l)$$

$$\frac{\partial C}{\partial b^l} = \delta^l, \quad \frac{\partial C}{\partial W^l} = \delta^l (a^{l-1})^T$$

- ▶ Start at the output, go *backwards*. Each δ^l reuses δ^{l+1} .
- ▶ Weight gradient = error at the node it goes *into* $\times$ signal from the node it comes *from*.
- ▶ The error is multiplied by f' in every layer.

Hidden activation functions



Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
 $\sigma' = \sigma(1 - \sigma) \leq \frac{1}{4}$.
Flat ($\sigma' \approx 0$) for large $|z|$.

tanh
 $\tanh' = 1 - \tanh^2 \leq 1$.
Centred at zero; also flat for large $|z|$.

ReLU $\max(0, z)$
 $f' = 1$ for $z > 0$, 0 otherwise. A unit with $z < 0$ for all inputs never learns (“dead”).

What matters

f must be **non-linear**: otherwise the whole network is a single linear map. For training, f' matters. Backpropagation multiplies by f' once per layer, so with L sigmoid layers the error shrinks by up to $(\frac{1}{4})^L$: the *vanishing gradient*. This is why ReLU is the default in deep networks.

Initialising the weights

Not all equal (e.g. all zero): then every unit in a layer computes the same thing, gets the same gradient, and stays identical. Random weights break the symmetry.

The scale matters. With M inputs per node and weights of variance σ_w^2 :

$$\text{Var}(z) \approx M \sigma_w^2 \overline{a^2}$$

- ▶ σ_w too large: $|z|$ large, tanh/sigmoid saturate, gradients blow up or die.
- ▶ σ_w too small: the signal shrinks in every layer, and so do the gradients.

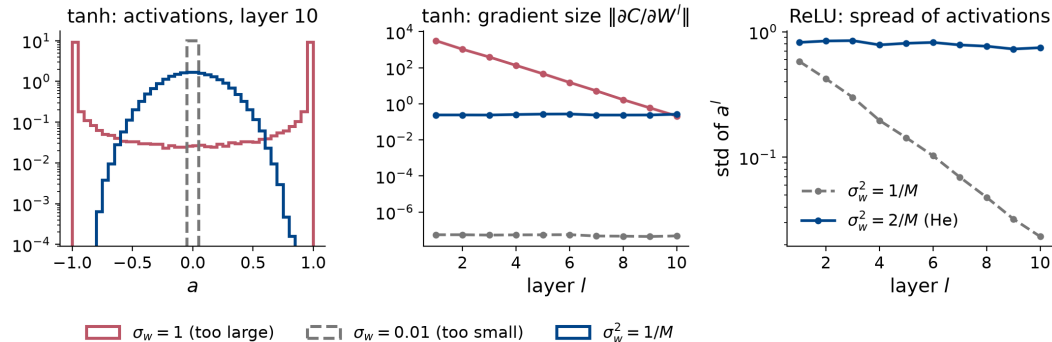
Rule: keep $\text{Var}(z)$ the same in every layer

$$\sigma_w^2 = \frac{1}{M} \text{ (tanh, sigmoid)}, \quad \sigma_w^2 = \frac{2}{M} \text{ (ReLU)}$$

ReLU sets half of its inputs to zero, hence the factor 2 (“He initialisation”).

The right scale depends on the *activation function*, and it decides whether *gradients* survive the backward pass.

Initialisation in a deep network



A network with 10 hidden layers of 256 units, random inputs, before any training.

- ▶ **Left:** with $\sigma_w = 1$ almost all tanh units sit at ± 1 (saturated); with $\sigma_w = 0.01$ everything is close to 0.
- ▶ **Middle:** the gradients grow by a factor 10^4 from layer 10 to layer 1 ($\sigma_w = 1$), or are tiny everywhere ($\sigma_w = 0.01$). With $\sigma_w^2 = 1/M$ they are the same size in every layer.
- ▶ **Right:** for ReLU, $1/M$ is not enough; $2/M$ keeps the signal alive.

Output activation and cost function come in pairs

Task	Output activation	Cost (one sample)	δ^L
regression, $y \in \mathbb{R}$	linear, $a = z$	$\frac{1}{2}(a - y)^2$	
binary, $y \in \{0, 1\}$	sigmoid, $a = \sigma(z)$	$-y \ln a - (1 - y) \ln(1 - a)$	
K classes, $\mathbf{y}$ one-hot	softmax, $a_k = \frac{e^{z_k}}{\sum_l e^{z_l}}$	$-\sum_k y_k \ln a_k$	

- ▶ The **output activation** gives the right *range*: any real number, a probability, or K probabilities that sum to one.
- ▶ The **cost** matches it: each is the negative log-likelihood of a Gaussian, Bernoulli or categorical distribution.

On the board (W2)

Compute $\delta^L = f'(z) \frac{\partial C}{\partial a}$ for the sigmoid with cross entropy.

Output activation and cost function come in pairs

Task	Output activation	Cost (one sample)	δ^L
regression, $y \in \mathbb{R}$	linear, $a = z$	$\frac{1}{2}(a - y)^2$	$a - y$
binary, $y \in \{0, 1\}$	sigmoid, $a = \sigma(z)$	$-y \ln a - (1 - y) \ln(1 - a)$	$a - y$
K classes, $\mathbf{y}$ one-hot	softmax, $a_k = \frac{e^{z_k}}{\sum_l e^{z_l}}$	$-\sum_k y_k \ln a_k$	$\mathbf{a} - \mathbf{y}$

- ▶ The **output activation** gives the right *range*: any real number, a probability, or K probabilities that sum to one.
- ▶ The **cost** matches it: each is the negative log-likelihood of a Gaussian, Bernoulli or categorical distribution.

On the board (W2)

Compute $\delta^L = f'(z) \frac{\partial C}{\partial a}$ for the sigmoid with cross entropy.

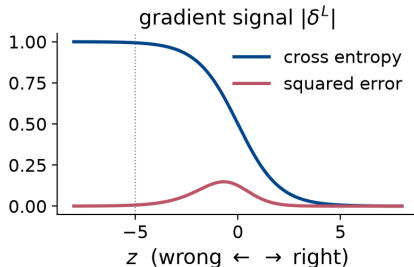
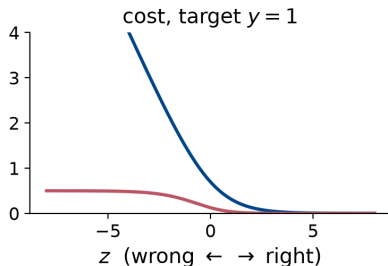
The connection

f' cancels against $\partial C / \partial a$:

$$\delta^L = \mathbf{a}^L - \mathbf{y}$$

prediction minus target, for all three pairs.

What goes wrong with a mismatched pair: sigmoid + squared error



Without the cancellation:

$$\delta^L = (a - y) \sigma'(z) = (a - y) a(1 - a).$$

Target $y = 1$, but $z = -5$: confidently wrong,
 $a = 0.0067$.

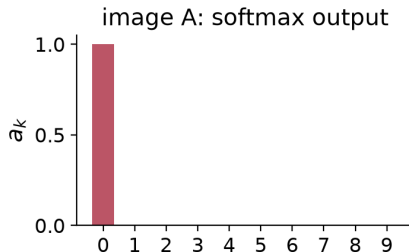
cross entropy: $\delta = -0.993$

squared error: $\delta = -0.0066$, **150 times smaller**.

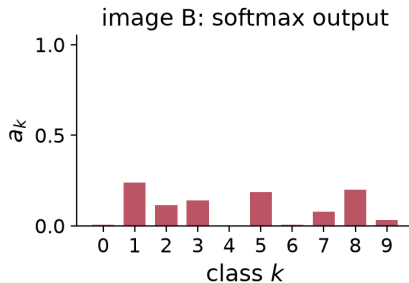
With the squared error, the more confidently wrong the network is, the *less* it learns. With the cross entropy it learns the most.

Quick question 1: which image trains the network?

label 0



label 1

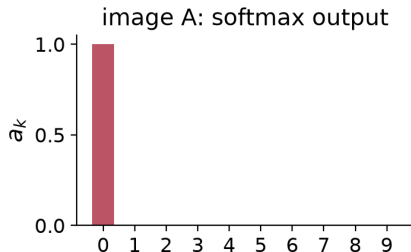


Question

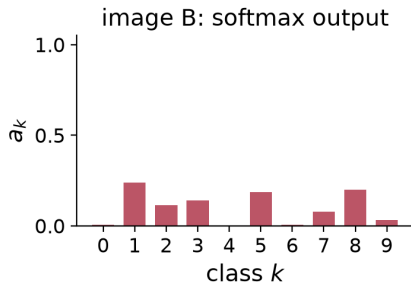
A network trained on handwritten digits (softmax + cross entropy) sees images A and B. Using $\delta^L = \mathbf{a}^L - \mathbf{y}$: which image changes the weights more?

Quick question 1: which image trains the network?

label 0



label 1



Question

A network trained on handwritten digits (softmax + cross entropy) sees images A and B. Using $\delta^L = \mathbf{a}^L - \mathbf{y}$: which image changes the weights more?

Answer

A: $\delta^L \approx \mathbf{0}$, nothing left to learn.

B: $\delta_1 = 0.24 - 1 = -0.76$, and $+0.20$, $+0.19, \dots$ for the other classes. Raise the score of class 1, lower the rest.

The network learns from what it gets wrong or is unsure about.

The universal approximation theorem

In words

A network with **one hidden layer** and a **non-linear** (non-polynomial) activation can approximate **any continuous function** on a bounded domain **as accurately as we want**, if the hidden layer is wide enough.

What it says

- ▶ A network *can* represent the function we are after.
- ▶ The non-linearity is essential.

What it does *not* say

- ▶ *How many* units are needed.
- ▶ Whether training *finds* the weights.
- ▶ How well it generalises to new data.

Why deep networks, then? Depth can do the same job with far fewer units.

Why it works, in one dimension: ReLU units are kinks

One ReLU unit $v \text{ReLU}(wx + b)$ is zero, then a straight line: one *kink*, at $x = -b/w$.

On the board (W3)

Build a “hat” from three ReLU units. Then build any curve from hats.

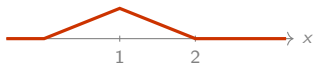
Why it works, in one dimension: ReLU units are kinks

One ReLU unit $\nu \text{ReLU}(wx + b)$ is zero, then a straight line: one *kink*, at $x = -b/w$.

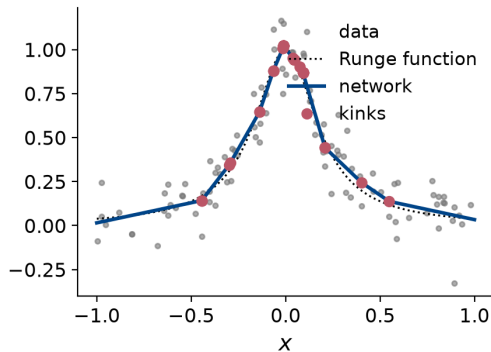
On the board (W3)

Build a “hat” from three ReLU units. Then build any curve from hats.

$$\text{ReLU}(x) - 2 \text{ReLU}(x - 1) + \text{ReLU}(x - 2)$$



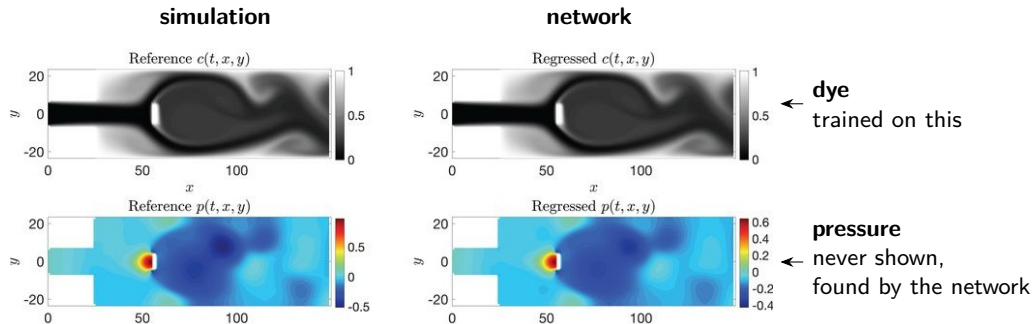
Hats at N points = connect-the-dots. More units, smaller error.



Training finds this by itself: a network with 50 ReLU units fitted to noisy data from the Runge function. The output is piecewise linear, and the kinks (red) sit where the function bends.

Motivation: seeing the invisible

We can film dye moving in a flow. Can a network work out the *pressure*?



Application: blood flow in brain aneurysms. Pressure and stress on the vessel wall from a medical scan, with no probe inside the patient.

M. Raissi, A. Yazdani, G. E. Karniadakis, *Hidden fluid mechanics*, Science **367**, 1026 (2020).

A network without data: the equation is the cost

Problem: $g'(x) = -\kappa g(x)$, $g(0) = g_0$. Nobody gives us data $g(x_i)$, only the equation.

Three ingredients:

1. A **trial solution** that satisfies the conditions for *any* network N :

$$g_t(x) = h(x) + B(x) N(x),$$

h satisfies the conditions, $B = 0$ where they are imposed.

2. The **residual**: $r(x) = g'_t(x) + \kappa g_t(x)$, zero for the exact solution.
3. The **cost**: the mean squared residual at points x_i we choose ourselves (*collocation points*),

$$C = \frac{1}{n} \sum_{i=1}^n r(x_i)^2.$$

What is new

The residual contains g'_t , so we need $\partial N / \partial x$: a derivative with respect to the *input*, not the weights. In code: automatic differentiation (`jax.grad` with respect to x).

What is not new

Network, optimiser and backpropagation for the weights. It is regression with “target” $r = 0$.

Example: exponential decay

$g' = -\kappa g$, $g(0) = g_0$, with $\kappa = 2$, $g_0 = 10$. Exact solution: $g = g_0 e^{-\kappa x}$.

On the board (W4)

Build g_t , compute g'_t , write the residual and the cost.

Example: exponential decay

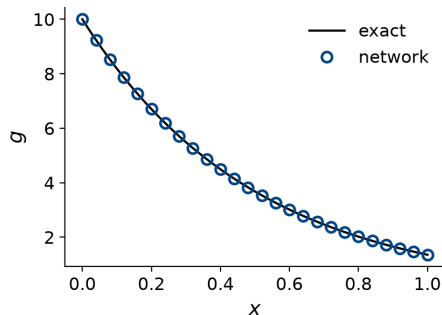
$g' = -\kappa g$, $g(0) = g_0$, with $\kappa = 2$, $g_0 = 10$. Exact solution: $g = g_0 e^{-\kappa x}$.

On the board (W4)

Build g_t , compute g'_t , write the residual and the cost.

$$g_t = g_0 + x N(x), \quad r = \underbrace{N + x N'}_{g'_t} + \kappa (g_0 + x N)$$

$g_t(0) = g_0$ whatever the network does.



Two hidden layers of 40 tanh units, 50 collocation points: max error 3×10^{-3} .

The activation function matters even more now

Poisson, diffusion and wave equations contain *second* derivatives, so we need $N''(x)$.

On the board (W5)

Differentiate a one-hidden-layer network

$N(x) = \sum_j v_j f(w_j x + b_j) + c$ twice with respect to x .

The activation function matters even more now

Poisson, diffusion and wave equations contain *second* derivatives, so we need $N''(x)$.

On the board (W5)

Differentiate a one-hidden-layer network

$N(x) = \sum_j v_j f(w_j x + b_j) + c$ twice with respect to x .

$$N''(x) = \sum_j v_j w_j^2 f''(w_j x + b_j)$$

ReLU: $f'' = 0$, so $N'' = 0$

A ReLU network is piecewise linear: *no curvature*, for any weights. It cannot solve $-g'' = f$.

$\max |N''|$ of a randomly initialised network:

tanh	2.9
GELU	1.2
sigmoid	0.06
ReLU	0

We use tanh: smooth, and $f' = 1 - f^2$,
 $f'' = -2ff'$ are cheap.

Boundary conditions built into the trial solution

Recipe: $g_t = h(x) + B(x) N(x)$, where h satisfies the conditions and $B = 0$ wherever a condition is imposed.

On the board (W6)

Build g_t for each set of conditions, and check them.

Boundary conditions built into the trial solution

Recipe: $g_t = h(x) + B(x) N(x)$, where h satisfies the conditions and $B = 0$ wherever a condition is imposed.

On the board (W6)

Build g_t for each set of conditions, and check them.

Conditions	Trial solution g_t
$g(0) = g(1) = 0$	$x(1-x) N$
$g(0) = a, g(1) = b$	$a + (b-a)x + x(1-x) N$
$g(0) = g_0, g'(0) = v_0$	$g_0 + v_0 x + x^2 N$

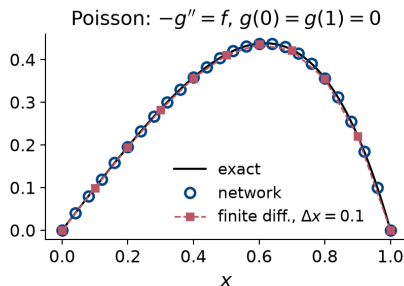
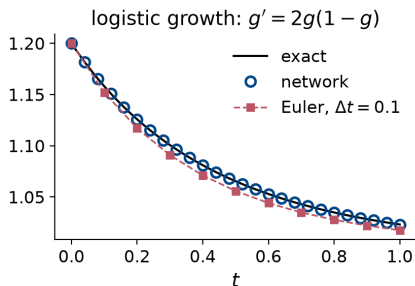
The good

The conditions hold *exactly*, for any weights: at the start, and after every step. The optimiser only has to solve the equation.

The bad

g_t must be designed by hand for every problem. A condition on the derivative, like $g'(1) = 0$, or a curved domain: hard or impossible.

Results against standard methods



Logistic growth is a *non-linear* equation, but has the same trial solution $g_0 + xN$; only the residual changes.

Logistic, max error		Poisson, max error	
Euler, $\Delta t = 0.01$	9×10^{-4}	FD, $\Delta x = 0.01$	2×10^{-5}
network	1×10^{-4}	network	1×10^{-5}

Read it honestly

The network is accurate. But Euler and finite differences take *microseconds*, the network thousands of training steps. For simple equations like these the standard methods win.

A partial differential equation: diffusion

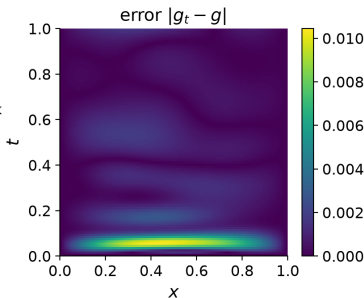
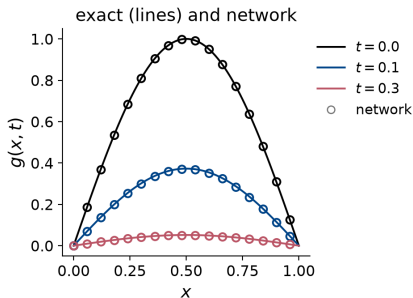
$$\frac{\partial g}{\partial t} = \frac{\partial^2 g}{\partial x^2}, \quad g(0, t) = g(1, t) = 0, \\ g(x, 0) = \sin \pi x.$$

The network now has *two inputs*, $N(x, t)$.
Nothing else changes.

Three conditions, one trial solution:

$$g_t = (1 - t) \sin \pi x + x(1 - x) t N(x, t)$$

Check: $t = 0$ gives $\sin \pi x$; $x = 0$ and $x = 1$ give 0.



The error is exactly zero on the three edges (built into g_t), and largest near $t = 0$, where the solution changes fastest: max error 1×10^{-2} . A standard finite-difference scheme gets 1.5×10^{-3} , in milliseconds.

Soft constraints: physics-informed neural networks (PINNs)

Instead of building g_t by hand: let the network *be* the solution, $u \approx N(x, t)$, and add the conditions to the cost as penalties:

$$L = L_{\text{PDE}} + \lambda(L_{\text{IC}} + L_{\text{BC}})$$

$$L_{\text{PDE}} = \frac{1}{n} \sum_i (\partial_t N - \partial_x^2 N)^2 \quad \text{inside the domain}$$

$$L_{\text{IC}} = \frac{1}{n} \sum_i (N(x_i, 0) - \sin \pi x_i)^2 \quad \text{at } t = 0$$

$$L_{\text{BC}} = \frac{1}{n} \sum_i (N(0, t_i)^2 + N(1, t_i)^2) \quad \text{at } x = 0, 1$$

A new condition is just a new term. λ weighs the terms, like the penalty in Ridge regression.

	RMSE	error at $t = 0$
hard (g_t)	1.8×10^{-3}	0 exactly
soft, $\lambda = 10$	6.5×10^{-3}	6×10^{-3}

Hard or soft?

- ▶ If you *can* build a trial solution, do: exact and more accurate.
- ▶ Soft is flexible, but the conditions hold only approximately.
- ▶ λ must be tuned. Too large, and the optimiser neglects the equation.

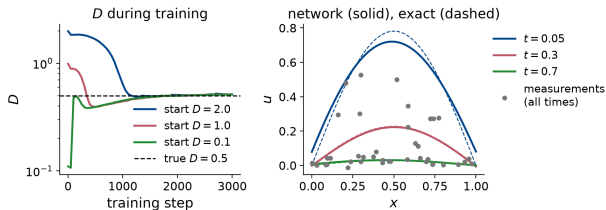
Why soft constraints at all? Inverse problems

$\partial_t u = D \partial_x^2 u$ with D **unknown**. No initial or boundary conditions, only 40 noisy measurements y_i .

Treat D as one more trainable parameter:

$$L(N, D) = \frac{1}{n} \sum_j (\partial_t N - D \partial_x^2 N)_j^2 + \lambda \frac{1}{40} \sum_i (N(x_i, t_i) - y_i)^2$$

No trial solution is possible here: soft constraints are the only option.



True $D = 0.5$. Three different starting guesses all give $D \approx 0.51$. **But** with five times more noise: $D = 0.69$. A recovered parameter needs an error bar.

Neural networks versus standard methods

Advantages

- ▶ **No mesh:** complicated geometries are easy.
- ▶ **High dimensions:** a mesh grows exponentially with the dimension, random points do not.
- ▶ **The solution is a function:** evaluate and differentiate it anywhere.
- ▶ **Non-linear equations** need no extra work.
- ▶ **Data and equations together:** unknown parameters, inverse problems.

Disadvantages

- ▶ **Slow:** thousands of training steps vs. microseconds.
- ▶ **No error guarantees** like “error $\propto \Delta x^2$ ”.
- ▶ **Local minima:** can converge to a wrong answer.
- ▶ **Hyperparameters:** architecture, learning rate, λ .
- ▶ **Smooth activations needed:** no ReLU for second-order equations.

Bottom line: simple problems in one or two dimensions: use standard methods. Networks pay off in high dimensions, complicated geometries and inverse problems. **Always compare with something.**

Summary

- ▶ **Backpropagation** is the chain rule: compute the errors δ' from the output backwards; weight gradient = error $\times$ incoming signal.
- ▶ **Hidden activations** must be non-linear; training sees f' .
- ▶ **Output activation and cost come in pairs**, and then $\delta^L = a^L - y$.
- ▶ **Universal approximation**: a wide enough network can represent any continuous function, but the theorem says nothing about size or training.
- ▶ **Differential equations**: the residual is the cost. Conditions go into the trial solution (hard) or the cost (soft). Use smooth activations. Flexible, but slower than standard methods and without their guarantees.

Next

Tuesday: backpropagation in code. **Exercises week 42:** Friday October 16. **Project 2:** November 6.